

Plan del TFG y de Shader DSL

Este plan cubre dos trabajos conectados: portar y validar el análisis de cromatina en GPU, y convertir Shader DSL y su extensión en herramientas reutilizables para otros proyectos. No presupone que una etapa implementada este ya validada. **Extraer** significa convertir código existente en una operación reutilizable; **portar** significa incorporar una funcionalidad presente en la referencia Python cuyo equivalente completo no aparece en C23.

Punto de partida

La referencia funcional está en `backups/python/CodigoPython/`, principalmente `main.py`, `trajectory_analysis.py`, `utils.py` y `fpsolve.py`. El pipeline GPU compartido está en `src/parseTextC23.shaderdsl.ts`; C2 analiza X y C3 analiza Y.

C23 ya contiene momentos, mínimos cuadrados, operador adjunto, Krylov/Arnoldi, Nelder-Mead, PDF estacionaria y puntuación de candidatos. Su `tauModelKL` compara distribuciones del modelo: **no reemplaza** el test de Markov de Python, que construye distribuciones de tres tiempos mediante KDE y usa sus divergencias para proponer lags. Python usa `dt = 1/253.2`; C23 parte de `dtSample=1.0`. Hay que igualar las unidades antes de comparar resultados.

La metodología de momentos a tiempo finito y Fokker-Planck adjunta tiene una referencia en [Callaham y colaboradores](#).

1. Datos y coordenadas

1. Portar la lectura de `Expression1` de archivos `.mat` a una entrada configurable del proyecto generado.
2. Rechazar entradas cuya forma no sea `[cromatinas, tiempos, 2]`, indicando la forma recibida.
3. Pasar frecuencia de muestreo y escala espacial junto a cada trayectoria, en vez de dejarlas implícitas.
4. Sustituir el `dtSample=1.0` de C23 por el intervalo físico correspondiente a los datos cargados.
5. Conservar el identificador de cromatina en todas las salidas de C2 y C3.
6. Permitir elegir coordenadas XY originales o PCA para una ejecución completa.
7. Igualar la PCA por trayectoria de `main.py`: centrado, dos ejes ortogonales y proyección de todos los puntos.
8. Guardar matriz de covarianza, ejes, valores propios y convención de signo usados en cada PCA.

2. Descripción de una trayectoria

9. Reproducir el histograma de 64 bins simétricos de `trajectory_analysis.py`.
10. Normalizar el histograma como densidad, conservando sus bordes y centros.
11. Mostrar el mapa XY de una cromatina coloreado por tiempo, como en `plot_chromatin_map`.
12. Añadir al mapa el centro y las dos direcciones principales de su PCA.
13. Portar la autocorrelación por FFT de `autocorr_func_1d`.
14. Marcar en la ACF el primer cruce por debajo de `0.5` y su tiempo físico.
15. Reproducir el ajuste exponencial de la ACF sin tomar logaritmos de valores no positivos.
16. Exportar los valores de PDF y ACF además de sus gráficas, para compararlos numéricamente con Python.

3. KDE del test de Markov

17. Generar la misma lista unica de lags enteros de `logspace(0, 2.5, 300)`.
18. Construir para cada lag las ternas $X(t)$, $X(t+\tau)$ y $X(t+2\tau)$ con identica longitud.
19. Rechazar lags que dejen demasiado pocas ternas e informar cuantas quedan.
20. Construir la malla comun de 50 puntos con el 20 % de margen usado por `markov_test_kde`.
21. Portar la eleccion Silverman de ancho de banda para datos de una, dos y tres dimensiones.
22. Calcular p_2 , la KDE unidimensional del estado intermedio.
23. Calcular p_{12} y p_{23} , las KDE bidimensionales de las dos transiciones.
24. Calcular p_{123} , la KDE tridimensional observada, con un limite explicito de memoria.

4. Divergencias y eleccion del lag

25. Normalizar p_2 , p_{12} , p_{23} y p_{123} con el volumen correcto de cada celda.
26. Construir $p(x_3|x_2)$ a partir de p_{23}/p_2 , protegiendo las zonas de densidad casi nula.
27. Construir $p_{123_markov} = p_{12} * p(x_3|x_2)$ y volver a normalizarlo.
28. Calcular $D_{KL}(p_{123} || p_{123_markov})$.
29. Calcular la divergencia inversa $D_{KL}(p_{123_markov} || p_{123})$.
30. Calcular la divergencia simetrica y conservar las tres curvas por separado.
31. Reproducir el suavizado gaussiano y la busqueda de minimos locales de Python.
32. Ofrecer seleccion del primer minimo, minimo global o todos los minimos; no reducir siempre el resultado a un unico `bestTau`.

5. Momentos y semilla del modelo

33. Crear las `stride` subsecuencias $X[\text{offset}::\text{stride}]$ para cada lag seleccionado.
34. Asociar a cada subsecuencia su tiempo real `tau_coarse = stride * dt`.
35. Igualar los momentos condicionales por bin: $E[\Delta X]/\tau$ y $E[(\Delta X)^2]/(2*\tau)$.
36. Conservar recuentos por bin y marcar los bins vacios como invalidos.
37. Igualar los errores por bin de drift y difusion que Python usa para ponderar el coste.
38. Reproducir la base inicial de drift $[1, x, x^2, x^3]$ y la de amplitud de ruido $[1]$.
39. Comparar la semilla de minimos cuadrados de C23 con `numpy.linalg.lstsq` para las mismas subsecuencias.
40. Permitir cambiar las bases del modelo desde un modulo, sin duplicar shaders para cada grado polinomico.

6. Ajuste AFP en GPU

41. Convertir la construccion de $f(x)$, $s(x)$ y $a(x)=s(x)^2/2$ en una operacion reutilizable.
42. Comparar las derivadas de borde y de interior de GPU con `AdjFP.derivs1d`.
43. Comparar el operador adjunto construido en GPU con `diag(f)Dx + diag(a)Dxx` de Python.
44. Comparar la accion de Krylov/Arnoldi con `expm(L*tau)` en mallas pequenas.
45. Comparar los momentos propagados m_1 y m_2 con `AdjFP.solve`.
46. Mostrar por separado el error ponderado de momentos y la penalizacion KL del coste.

- 47. Registrar en Nelder-Mead reflexion, expansion, contraccion, *shrink* y motivo de parada.
- 48. Conservar el mejor resultado valido y un motivo explicito de *fallback* cuando falle AFP.

7. PDF estacionaria y candidatos

- 49. Comparar el solver estacionario GPU con `SteadyFP.solve` para modelos sencillos.
- 50. Guardar masa total, minimo de la PDF y residuo estacionario de cada candidato.
- 51. Rechazar candidatos con PDF negativa fuera de tolerancia, como hace `select_best_models`.
- 52. Extraer el KL entre PDF experimental y PDF estacionaria como modulo reutilizable.
- 53. Conservar coste, KL, validez y coeficientes para **cada** pareja (`lag`, `subsecuencia`).
- 54. Reproducir la seleccion del porcentaje superior de candidatos sin descartar el resto de resultados.
- 55. Mostrar conjuntamente el mejor candidato automatico y los candidatos elegibles de otros minimos de lag.
- 56. Exportar los coeficientes de drift y amplitud de ruido de los candidatos seleccionados, como `save_polynomial_coefficients`.

8. Modulos numericos reutilizables

- 57. Extraer de C23 un modulo `ConditionalMoments` para momentos por bin y lag.
- 58. Crear `MarkovLagScan` para la cadena ternas, KDE, divergencias y minimos.
- 59. Crear `KLDivergence` para escribir, por ejemplo, `k1 = KLDivergence p=pdfObservada q=pdfModelo`; la sintaxis exacta puede cambiar.
- 60. Extraer un modulo `StationaryFP` con sus modos de contorno y sus indicadores de validez.
- 61. Extraer `AdjointFP` para operador, momentos y propagacion a tiempo finito.
- 62. Extraer `NelderMeadGPU` para reutilizar la minimizacion fuera del caso de cromatina.
- 63. Anadir una operacion de reduccion GPU para sumas, minimos, maximos y recuentos sin leer texturas completas en CPU.
- 64. Anadir convolucion gaussiana reutilizable; que la utilicen tanto el suavizado de KL como futuros filtros de datos.

9. Extension: inspeccion numerica

- 65. Mostrar en cada draw el nombre de la etapa cientifica y que entradas y salidas consume.
- 66. Abrir desde el inspector la textura de momentos de un lag y una subsecuencia concretos.
- 67. Mostrar en una misma vista `f_KM`, `a_KM`, errores y curvas reconstruidas `f`, `a`.
- 68. Mostrar por separado las tres curvas KL del test de Markov, sin confundirlas con `tauMode1KL`.
- 69. Senalar en esas curvas los minimos candidatos y el lag finalmente elegido.
- 70. Anadir escalas numericas y marcas de NaN/infinito a las imagenes de texturas.
- 71. Comparar dos iteraciones de backup con diferencia absoluta y relativa por canal.
- 72. Vincular cada valor de coste mostrado a su etapa de calculo y a los uniforms que lo produjeron.

10. Extension: seguimiento del pipeline

- 73. Mostrar la procedencia de una textura: programa productor, draw, framebuffer e iteracion.
- 74. Mostrar los consumidores de esa textura antes y despues de un `rebind-temp`.

75. Detectar una lectura y escritura incompatibles sobre el mismo recurso en una pasada.
76. Advertir si un programa usa `dt` en muestras y otro en segundos.
77. Indicar que import y que snippet introdujeron una variable utilizada en C23.
78. Ofrecer navegacion desde `tauBest` hasta los costes y mascaras que determinaron su seleccion.
79. Mantener una vista de ejecucion por cromatina/eje, evitando mezclar backups de C2 y C3.
80. Reducir lecturas repetidas de backups durante `recomputeTau`, conservando solo lo necesario para la vista abierta.

11. Comprobaciones cuantitativas

81. Usar una trayectoria sintetica Browniana para comprobar la difusion sin drift.
82. Usar Ornstein-Uhlenbeck para comprobar recuperacion de drift y densidad estacionaria conocidos.
83. Usar un potencial de doble pozo para probar una PDF no unimodal.
84. Usar ruido temporalmente correlacionado como control donde el test de Markov no deberia dar la misma respuesta.
85. Comparar Python y GPU etapa por etapa: PDF, KDE, momentos, semilla, AFP, PDF final y ranking.
86. Repetir esas comparaciones con distintas precisiones y numeros de bins.
87. Medir tiempo y memoria de Python frente a GPU para una y varias cromatinas.
88. Separar desacuerdo numerico aceptable de diferencias que cambien el lag o el modelo elegido.

12. Resultado del TFG

89. Ejecutar el pipeline completo para cada cromatina disponible, primero en X y despues en Y.
90. Repetirlo en XY original y PCA para comprobar cuanto cambia la interpretacion.
91. Conservar por cromatina las curvas de Markov, los lags candidatos y el elegido.
92. Conservar por cromatina los campos de drift y difusion y sus PDFs observada/modelada.
93. Representar la relacion entre coste AFP, KL estacionaria y seleccion final.
94. Comparar condiciones celulares solo cuando ambas esten disponibles con metadatos y muestreo compatibles.
95. Presentar una demostracion desde una declaracion breve del modulo hasta sus texturas y resultados inspeccionables.
96. Cerrar la memoria distinguiendo tres hechos: equivalencia con Python, aceleracion/claridad aportada por el DSL y conclusiones que realmente permiten los datos.

Ruta critica

Seguir primero **1-8 -> 17-32 -> 33-56 -> 81-88 -> 89-96**. Incorporar los modulos y las mejoras de la extension cuando simplifiquen o permitan verificar una etapa concreta, sin convertirlos en condicion para iniciar el estudio.